Local research workbench · UI MVP

Run the workflow. Check the evidence. Keep the limits visible.

A practical manual for setup, the first synthetic run, result review, and recovery. Follow one action at a time. Check the expected result before you continue.

First successful flight

Install the locked environment. Start the local server. Create a synthetic study. Run `heuristic-hu`. Open the HTML research report.

No patient data, model download, or AI server is needed for this first run.

Research use only. This is a self-made experimental tool created with AI assistance, to the best of our knowledge. It is not medical advice from a qualified healthcare professional, a diagnosis, or a medical device. Findings may be incorrect, incomplete or misleading. Do not use the output for diagnosis or treatment decisions.

Choose your checklist

1. Preflight: install and start
2. The browser workflow: input to offline report
3. Read findings, severity and review context
4. Repeat the workflow from the command line
5. Configure paths and local models
6. Set up models and understand wait estimates
7. Acquire a cohort and audit DICOM headers
8. Benchmark and resume development cases
9. Queue, stop and recover processing jobs
10. Review stored runs in Evaluation
11. Protect data, recover and shut down
12. Check visual evidence and heuristic context

Author: Marcel Petrick · GPL-3.0-or-later · Project generated with AI assistance.
Written in a flight-manual style with short actions and explicit checkpoints; no formal technical-English compliance claim. This manual describes the current commands. Sealed-holdout evaluation and the LAN runner are not implemented.

Install. Verify. Start locally.

Use a terminal on the computer that will run the software. The commands below assume a source checkout and `uv`. Replace the example path with your checkout path. If `uv` is missing, follow the official uv installation instructions first.

1. **Open the repository directory.**

   ```
   cd /home/mpetrick/repos/ThoraxGPT_from_Wish
   ```

2. **Install the locked Python environment.**

   ```
   uv sync --frozen --all-groups
   ```

   Python 3.14 or newer is required. `uv` can provision the requested Python environment. Initial installation needs package access; inference stays local.

3. **Check the installed command.**

   ```
   uv run thoraxbench --version
   uv run thoraxbench --help
   ```

   Expect a version, the research disclaimer, and the command list.

4. **Start the workbench.**

   ```
   uv run thoraxbench serve
   ```

   Leave this terminal open. Open http://127.0.0.1:8000 on the same computer.

> **Checkpoint:** The “Analyze one study” page appears in English and light mode. The page shows the research disclaimer. A new installation may have no studies yet. This is expected.

Required for the demo

A supported Python environment, the locked dependencies, a browser, and enough free disk space for synthetic data and outputs.

Optional for vision models

A local Ollama installation and compatible model weights. The deterministic `heuristic-hu` baseline works without them.

Port 8000 is already in use

```
uv run thoraxbench serve --port 8001
```

Open `http://127.0.0.1:8001`. The server binds to loopback. LAN access and remote inference are not supported in this MVP.

Complete your first browser run.

Create demo → **Inspect study** → **Run locally** → **Review** → **Export**

1. **Select “Create synthetic demo”.**
   Expect a synthetic study with 64 axial slices. It contains simulated nodule and fluid-density regions. The images contain no patient data.
2. **Inspect the study viewer.**
   Move the slice slider. Use the Left and Right arrow keys when the slider has focus. Change the display window to Lung, Mediastinum, Bone or Abdomen. Window changes affect the viewer only.
3. **Select “heuristic-hu” as the model.**
   This baseline uses HU-volume measurements. It sends no rendered images to a vision model. The image-count control does not affect its predictions.
4. **Select “Run analysis locally”.**
   Wait for a completed result or explicit failure/abstention. “Job elapsed (server clock)” shows time since the worker started, including loading, inference and reporting. It differs from recorded “Engine stages”. Expect synthetic nodule and effusion findings.
5. **Read the status before you read the findings.**
   Check study quality, the finding table, severity/review context and the English research report. Follow the interpretation checklist on the next page.
6. **Select “Download HTML”.**
   Open the saved file in a browser. Check the findings and embedded image gallery. Also save JSON for structured output and “Download report” for Markdown.

> **Checkpoint:** The HTML report opens after you stop the server. It needs no external images, fonts, scripts or internet access. Missing evidence references remain explicit.

Optional: compare the recommended local models

Enable “Compare recommended local models”, then select “Compare local models”. Available `heuristic-hu`, MedGemma Q4/Q8, Gemma 3 and Qwen3-VL run sequentially; unavailable models are skipped. “Batch progress” shows the current model, not inference percent. Save per-model downloads and “Download comparison HTML”. See model setup and rough wait estimates before a longer run.

Keyboard help and preferences

Use Tab to reach controls and help-bearing fields. Focus or hover shows translated help after a short delay; you can move the pointer onto it. Escape dismisses it until you move to another control. The language switch selects English or German. The theme switch selects light or dark. Both choices persist in this browser. Reports and prompts remain English. About lists the author, version and library licenses.

> **Privacy checkpoint:** A downloaded report contains images and research identifiers. Keep real-study exports in controlled local storage. The browser may save downloads outside the repository; manage those copies as sensitive data.

Inspect claims before conclusions.

Check every claim against evidence. The software cannot establish health, care needs or a diagnosis.

Read first	What it means	Operator action
Completed	The engine returned output. This does not validate it.	Inspect QC, findings, evidence and provenance.
Abstained / failed	No usable research reading is available.	Read the reason. Preserve the failed run. Do not interpret it as “nothing found”.
Present / not detected / uncertain	The model's reported presence status.	Treat negative and uncertain output as fallible.
Presence score	Uncalibrated support for presence; not measured accuracy.	Do not treat 0.79 as a validated 79% probability.
Severity / typical attention	Educational reference context, not validated triage.	Check its assumptions and sources. Do not derive personal medical instructions.

Severity and “should this be checked?”

The reference scale runs from **incidental** → **low** → **moderate** → **high** → **critical**. Typical attention uses **routine, soon, urgent, emergency**. These describe general reference knowledge for a reported label. They do not establish the severity or urgency of an actual person's condition.

> **No automatic clearance:** An absent, low-rated, unknown or uncertain finding does not establish that no professional review is needed. The tool cannot answer whether a person should be checked. Its output is not advice from a healthcare practitioner.

Check image evidence

- Vision-model reports show the actual prepared input images. **Image 1** is the first model image. A sampled image set may miss other parts of the study.
- Heuristic reports show **illustrative source-slice previews**. Their source indices start at **0**; they are not vision-model inputs.
- An evidence link is a reported association, not verified localization or a segmentation. Invalid or missing references must remain unresolved.

Compare models without implying consensus

Compare each model column by finding, status and uncalibrated score. Keep abstentions, failures and skipped models visible. Agreement is not validation; disagreement is not an accuracy ranking. Use each model’s downloads to inspect its own images and provenance. A combined offline HTML report preserves the comparison and individual evidence reports.

Check reproducibility

Save JSON for original responses, QC, hashes and provenance. “Job elapsed (server clock)” covers processing time for the job; “Current model elapsed” times the active model. “Engine stages” exclude loading and reporting. Timers do not measure inference progress. Synthetic runs and transport tests do not establish patient accuracy.

Repeat a run and save its artifacts.

Run these commands from the repository root. Keep the working directory unchanged between steps. These examples use only synthetic data.

1. **Create a fresh synthetic study.**

```
uv run thoraxbench demo-data data/demo/manual
```

2. **Run the deterministic baseline.**

```
uv run thoraxbench analyze data/demo/manual \
  --model heuristic-hu \
  --output data/results/manual/result.json \
  --report data/results/manual/report.md \
  --html data/results/manual/report.html
```

3. **Open the saved HTML file.**

   Open `data/results/manual/report.html` in your browser. Inspect the matching JSON and Markdown files in the same folder.

> **Checkpoint:** A successful run returns exit status 0. A failed run or abstention returns a nonzero status. When a result exists, saved artifacts explain the abstention. Failures before result creation may produce only an error message.

Optional: run a local vision model

First check availability. A configured model is not necessarily installed, licensed for your intended use, or accurate.

```
uv run thoraxbench models
uv run thoraxbench analyze data/demo/manual \
  --model medgemma1.5-q4 --n 4 \
  --output data/results/manual/vision.json \
  --report data/results/manual/vision.md \
  --html data/results/manual/vision.html
```

The model must already exist in local Ollama. Larger image requests may take minutes. Empty predictions, truncated output, invalid responses and unsuitable inputs can cause abstention.

Available overrides

`--model` selects an ID from `configs/models/`. `--prompt` selects P0–P5 (P5: strict schema, `IMAGE k` captions, no heuristic context). `thoraxbench coverage` reports how many calibration reference nodules land on attached images. `--strategy` overrides the sampling strategy. `--n` requests a positive number of sampled slices; the browser limits it to 1–16. Use `uv run thoraxbench analyze --help` for the current options.

HTML output must be inside the configured data directory's `results/` folder. Destinations or symlinks that escape that folder are rejected before inference. JSON and Markdown examples also stay inside ignored results storage.

Keep paths explicit and inference local.

Set environment variables before starting the command or server. Existing data and manifests do not move when you change a setting.

Environment variable	Default / role
`THORAXBENCH_DATA_DIR`	`<checkout>/data`. Raw data, caches, splits, sealed IDs, demos and results. Keep medical data outside Git.
`THORAXBENCH_OLLAMA_URL`	`http://localhost:11434`. Only loopback inference endpoints are accepted.
`THORAXBENCH_DATA_BUDGET_GB`	`10.0` decimal GB. `thoraxbench acquire` enforces it for its own downloads. Analysis, demo and benchmark routes do not; inspect actual usage before writing.
`THORAXBENCH_CT_CLIP_URL` `THORAXBENCH_M3D_URL`	`http://localhost:8011` / `:8012`. Optional CT-CLIP / M3D services.

`THORAXBENCH_CT_CHAT_URL`
`THORAXBENCH_RADFM_URL`
`THORAXBENCH_HUATUO_URL`
	`http://localhost:8013` / `:8014` / `:8015`. Optional services; not installed by the MVP.

```
THORAXBENCH_OLLAMA_URL=http://127.0.0.1:11434 \
  uv run thoraxbench serve --port 8000
```

Model setup

1. **Inspect the model YAML.**

   Find its adapter, tag, view and prompt under `configs/models/`. Example: `medgemma1.5-q4` expects Ollama tag `medgemma1.5:4b-it-q4_K_M`.

2. **Check model terms and hardware needs.**

   See `docs/model_cards.md` and `docs/model_selection.md`. A model ID is not a download guarantee. Gated terms remain separate from dataset licenses.

3. **Start the local model service.**

   If Ollama is installed but not running, start it using your local installation's service controls or `ollama serve`. If it is already running, use that instance.

4. **Check the installed tags.**

   ```
   ollama list
   uv run thoraxbench models
   ```

Configuration files, prompts, preprocessing and sampling affect results. Preserve version control and the recorded hashes. Remote/LAN inference is a design proposal only; do not bypass the loopback guard.

Install explicitly. Expect variable waits.

Select the unavailable model in “Model”, then select “Set up local models”: the setup table opens at that model’s row, and Run stays disabled until it is available. Opening setup starts no download. Alternatively open **Model availability & setup**. Check availability, adapter, license and provider information. “Available” means the local runtime responds; it does not establish accuracy, GPU capacity or useful predictions.

Check actual inputs: anatomy sampling preserves landmarks and can exceed the image target. Coronal views can add images. Inspect prepared previews/counts. Wait estimates use the requested count and remain guesses. A strict small-image selection uses uniform sampling without added views.

Normal CLI setup: automatic pull on run

After reviewing the publisher’s terms, run this from the repository root. Ollama downloads missing weights and opens chat:

```
ollama run medgemma1.5:4b-it-q4_K_M
```

Enter `/bye`. Run `ollama list`. Select **Refresh model availability**. Use the YAML tag, not app ID `medgemma1.5-q4`. For download only, use `ollama pull medgemma1.5:4b-it-q4_K_M`. See Ollama CLI reference. App analysis does not auto-pull.

Optional browser download

Prerequisites: running loopback Ollama, network access and enough space in its model store. Weights are stored by Ollama, outside this repository’s data folder.

1. **Open “Model terms”.**
   Check the publisher’s terms and any access requirements.
2. **Check “I reviewed and may comply with the applicable model terms”.**
   Acknowledge this only when the terms permit your use.
3. **Select “Download model weights”.**
   The app asks local Ollama to pull that configured tag. Downloads never start automatically.
4. **Inspect “Model download”.**
   “Active layer bytes” reports actual current-layer bytes. It can reset between layers; it is not a whole-model percentage. Analysis and setup controls remain locked while the download is pending.
5. **Wait for “Download finished”.**
   The inventory refreshes. Confirm availability before analysis.

> **Recovery:** for “Download failed”, inspect the error. Check terms/access, network, disk space and local Ollama. Retry with an explicit download action. The app has no download-cancel endpoint; closing the page does not confirm cancellation.

Set up an optional HTTP model service

Read the provider links and **Provider configuration (English)**. Obtain upstream access approval and permitted weights. Check hardware and dependencies. Start the provider’s compatible local service at its configured loopback URL, then select **Refresh model availability**. A healthy service can become available. The app does not install those services or bypass gated access.

Read the rough remaining estimate

Rough remaining estimate starts with workstation guesses: **1–60 seconds** for the HU baseline and **1–15+ minutes** for vision models. Later estimates use recent local request durations for the same model and requested image count. Only numeric duration history is saved in this browser; no study identifiers are stored with it.

A batch adds estimates for remaining available models. If the upper estimate is exceeded, the UI says **Taking longer than estimated; still running**. Hardware, input size, cold loading and generation can extend the wait. These are guesses, not measured benchmarks, guaranteed deadlines or inference-progress percentages.

Acquire under the gates. Audit the headers.

Start in the repository root. Check `uv run thoraxbench datasets`. Acquisition needs TCIA access and `approved*` status. The pilot cohorts are already acquired and split.

Acquire an approved TCIA collection

1. **Show the plan and the budget verdict.**

   ```
   uv run thoraxbench acquire <id> --dry-run --patients 20
   ```

   No download starts. Check counts, estimated size, usage and `fits`.

2. **Start the download with the same options.**

   ```
   uv run thoraxbench acquire <id> --patients 20
   ```

   To resume, re-run without `--patients`. Complete series are skipped.

3. **Re-hash the stored files.**

   ```
   uv run thoraxbench acquire <id> --verify
   ```

   After a split exists, only development patients are opened and hashed.

4. **Freeze the patient split before indexing.**

   Follow the recovery checklist.

> **Checkpoint:** exit 0. `data/raw/<id>/` contains series, checksums, acquisition manifest and license snapshot.

Observed problem	Recovery action
Exit 2: `Acquisition refused`	A license, split or budget gate failed. Resolve the cause; never edit status merely to pass.
`REFUSED: exceeds the data budget`	Lower `--patients` for a new selection, or free space.
Exit 1: download or checksum failure	Re-run later. For an incomplete or mismatched series, remove only that series directory and re-run.

Audit DICOM headers for identifiers

```
uv run thoraxbench privacy audit data/demo/manual
```

The audit prints tag keywords and file counts, never values. Add `--json` for structured output. Add `--strict` to exit with status 1 on a `fail` item.

> **Checkpoint:** the phantom reports `FAIL` for its synthetic `PatientName` and three review items. **Recovery:** resolve real-study identifier findings before analysis/sharing. The audit checks no pixels and does not prove anonymity.

Score development cases. Keep the holdout sealed.

Prerequisites: the datasets named in the experiment are split and indexed. Experiments live in `configs/experiments/`: E000 is the deterministic HU-heuristic baseline; E001–E004 need MedGemma in local Ollama.

1. **Run a quick check on a few cases per dataset.**

```
uv run thoraxbench benchmark \
  configs/experiments/E000_heuristic_hu_baseline.yaml --limit 2
```

2. **Run the full experiment.**

```
uv run thoraxbench benchmark \
  configs/experiments/E000_heuristic_hu_baseline.yaml
```

3. **Save the printed `Invocation:` path.**

   The manifest and checkpoint exist before inference. Terminal case/variant artifacts accumulate under `cases/`. Read the final `Report:` path and matching `result.json` when the run finishes. See the resume checklist.

4. **Compare the printed config SHA-256 with `docs/results.md`.**

   A different hash means the experiment file changed. Record a new run; do not overwrite the old entry.

Run folders use `<timestamp>-<hashprefix>-<uuid>`. Repeated or limited runs preserve earlier outputs, including legacy flat files. The experiment's `labels` select both requested and scored findings. Prompt precedence is variant, experiment, model configuration, then `P3`.

Checkpoint: the command prints status counts per variant, the result and report paths, and the config SHA-256. Only development cases are read. Failures and abstentions stay explicit per case and are excluded from label metrics.

Observed problem	Recovery action
Exit 1: `Benchmark failed`	Read the named cause, for example a missing split or index. Freeze and index the dataset first.
Exit 2 after `--unseal`	Expected. Sealed evaluation is not implemented. Run without `--unseal`.
Vision model unavailable	Check `uv run thoraxbench models` and local Ollama. Do not substitute a cloud endpoint.

Check MedGemma readiness before larger sweeps. From the repository root, run `uv run thoraxbench benchmark configs/experiments/E001_calibration_smoke.yaml --limit 1`. This separate smoke uses one calibration case per dataset, Q4/Q8, four 256-pixel axial inputs and two labels; no BF16 or bootstrap. It is not the full E001 experiment. If predictions are empty or truncated, inspect the calibration pipeline before expanding the run.

Read results with their limits: E000 measures the pipeline of a non-learned control, not model accuracy. Scores are uncalibrated. A `--limit` run is partial. No result here is a sealed-holdout evaluation or clinical validation.

Preserve the record. Resume pending work.

New benchmark runs save an invocation manifest and atomic checkpoint before inference. Keep that complete directory under ignored `data/results/`. Case JSON can contain sensitive research identifiers and raw text; do not commit it.

Artifact	What to inspect
`manifest.json`	Immutable experiment, selection, source/data/model/prompt and runtime identity.
`checkpoint.json`	Authoritative terminal-unit progress and integrity checksums.
`cases/*.json`	Resolved spec, request, outcome/report and available analysis: redacted raw output/error, QC, timings and provenance. Ollama metadata retains rendered text/settings and decoded provider responses; malformed text is bounded. Image bytes become hashes; HTTP-service capture remains limited.
`result.json` / `report.md`	Final aggregates. Historical flat files remain unchanged.

Continue an interrupted limited run

Prerequisites: the original new-format invocation, unchanged experiment/limit, code/commit, dependency lockfile, data/index/split, prompts/model settings and runtime weights. Vision/HTTP services must expose a known matching model digest. Identity checks may take time before progress appears.

1. **Locate the original `Invocation:` directory.**

   Use its final folder name in place of `PASTE_PRINTED_RUN_ID` below.

2. **Run with the same experiment and limit.**

   ```
   uv run thoraxbench benchmark \
     configs/experiments/E000_heuristic_hu_baseline.yaml \
     --limit 2 --resume data/results/benchmark/E000/PASTE_PRINTED_RUN_ID
   ```

3. **Inspect all statuses in the final report.**

   Resume runs only pending units. Checkpointed scored, failed, abstained and not-indexed outcomes remain terminal. Start a new invocation to retry failures.

> **Checkpoint:** a complete validated run returns existing aggregates without inference or rewriting. An interrupted in-flight unit may run again; a case file absent from the checkpoint may be an orphan and is not automatically reused.

Recovery: changed identity, invalid checksums, an unknown model digest, a symlink escape or an active writer refuses resume. Preserve the evidence. Resolve the cause or start a new invocation; never edit checksums to force reuse. Legacy runs without checkpoints cannot resume.

Compare methods before metrics. Missing presence/confidence remains unavailable. Zero available requested-label predictions abstain; partial coverage never invents negatives. Task E reports omissions and coverage. Task G excludes abstained/unknown decisions and exposes its eligible reassurance denominator. Current prompts use one-based image captions. Historical results were not recomputed under these changes.

Available dataset/split/reference hashes, scoring/schema source and input hashes (HU bytes/geometry plus prepared input) identify the setup. Missing sources remain unavailable; hashes do not validate labels. Selected direct header values and paths are redacted from output text/metadata, including cached-study analysis. This is not OCR or anonymity; source-unavailable caches cannot recover identifying header values.

Review stored runs. Start nothing.

Prerequisites: the local server runs (`uv run thoraxbench serve`) and at least one benchmark invocation finished. **Evaluation** only reads `data/results/benchmark/<experiment>/<run>/` . It never starts inference, writes files, re-renders images or reads sealed cases.

1. **Select “Evaluation” in the sidebar.**

   The runs table shows experiment, run folder, config hash, start time (UTC), state, variants and status counts. Your “Analyze one study” session, including a running request, stays intact.

2. **Select “Open” on a complete run.**

   Read each variant's stored metrics, then the case table: dataset, case, phase, variant, status, reference labels, prediction, abstention/parse warnings and evidence check.

3. **Narrow the rows.**

   Use “Status filter”, “Evidence filter” and “Variant filter”, for example to list every abstention or every unsupported positive.

4. **Select “Details” on a row.**

   Read reference versus prediction per label, the images sent to the model, the per-finding evidence check, the parsed JSON and the raw model response as plain text.

5. **Compare setups.**

   Check two or more setups, then select “Compare selected”. The table reports basis, differences, shared cases, paired available predictions and experimental factors. It never ranks models.

> **Checkpoint:** complete runs open; every other folder shows its state and reason. Benchmark artifacts store no prepared image pixels: the case detail states this and shows the recorded image count and captions instead.

State	Recovery action
Incomplete	Let the CLI run finish or continue it with `--resume` , then select “Refresh runs”.
Unavailable	Legacy, corrupt or non-checkpointed output. Preserve it; start a new invocation.
Refused	The stored selection fails the current development split guard. Do not edit split files to open it.

> **Read with limits:** scores are uncalibrated development measurements, not clinical accuracy. “Unavailable” predictions are not negatives. In-range evidence numbers are bounds-checked references, not proof of visual correctness. Heuristic-baseline references are zero-based source slices.

Stop safely. Keep the holdout sealed.

Before any real-study workflow

Verify the specific dataset's license status with `uv run thoraxbench datasets`. Acquisition requires an approved registry entry; never infer a license from the hosting site. Acquire the cohort first (page 08).

1. **Freeze the patient split before indexing or analysis.**

   ```
   uv run thoraxbench split lidc_idri
   ```

   Run this only for an approved, already downloaded cohort that has no split yet. Existing frozen splits refuse replacement. Do not regenerate them.

2. **Index only permitted development cases.**

   ```
   uv run thoraxbench index lidc_idri --limit 1
   ```

   Indexed development studies appear in the browser. The browser has no arbitrary-folder upload flow. The CLI can read a permitted study directory through its guards.

Never open `data/sealed/`, inspect holdout images, or copy sealed cases into development folders. No current command evaluates the holdout. `thoraxbench benchmark --unseal` is refused; sealed evaluation is not implemented.

Observed problem	Recovery action
Model unavailable	Open Model availability & setup. Check `ollama list`, the configured tag and loopback URL. Use `heuristic-hu` for the demo. Do not substitute a cloud endpoint.
Slow run / empty or invalid output	Preserve the error and result. Try a smaller synthetic vision request. Do not turn an abstention into a negative finding.
Study missing / guard rejection	Check the development split and index. A holdout rejection is intentional. Do not bypass it.
Disk full / cannot write	Check `df -h .` and `du -sh data`. Review disposable generated results/caches before removing anything. Preserve raw data, manifests and split files.
Old download link stops working	Download links are registered for the running server process. After restart, open the saved files under `data/results/ui/` directly or rerun the study.

Shutdown and handover

Press **Ctrl+C** in the server terminal. Completed result files remain on disk. Record the model, settings, version, result paths, warnings and next action before you hand work to another researcher. Never commit patient images, rendered CT slices, model weights or secrets.

Maintain this manual: edit `manual.html`, then regenerate the PDF from the repository root with `uv run python -m playwright pdf --paper-format A4 file://"$PWD"/manual.html manual.pdf`. Install Chromium with `uv run playwright install chromium` if needed. Check `docs/plan.md` for current scope; this manual contains no patient images.

Check the inputs. Keep uncertainty.

> **Research use only.** This self-made experimental tool may be incorrect, incomplete or misleading. It is not professional medical advice or a basis for diagnosis or treatment.

Understand which image number is cited

Vision-model evidence uses one-based attached-image numbers: Image 1 is the first prepared image. A source-slice position in a caption is not an attached-image number. The HU baseline instead uses zero-based source-slice indices for its explanatory previews.

New constrained Ollama runs with P2–P4 remove invalid or duplicate image references. A default-positive prediction (implied presence probability at least 0.5) without valid attached-image evidence becomes unavailable before scoring. If no requested predictions remain available, the run abstains. The raw response remains inspectable. An in-range number still does not prove visible support.

Separate visual output from heuristic context

The versioned multi-slice view parameter `include_cues` defaults to `true`. Set it to `false` in model YAML `view_params` or experiment `view_overrides` to omit heuristic prompt text. The images and sampling remain unchanged. Use YAML booleans only; null, numbers and strings are refused. View hashes and recorded context distinguish this ablation. Preprocessing still computes cues for sampling; this is not a speed comparison.

Prerequisites: locked environment, approved split/index, installed Q4 tag and local Ollama. Use calibration cases only. Run from the repository root:

```
uv run thoraxbench benchmark \
  configs/experiments/E001_cue_ablation.yaml --limit 1
```

1. **Keep the case bound.**

   The YAML does not enforce `--limit 1`. This selects at most one calibration case per dataset and runs two sequential variants.

2. **Inspect actual prepared-image counts.**

   Anatomy sampling retains landmarks and can exceed its target. Both variants must use identical image hashes.

3. **Read both responses and warnings.**

   Compare omitted labels, unsupported positives, failures and abstentions. Do not infer correctness from confidence or agreement.

4. **Preserve the invocation folder.**

   Requests, responses, provenance and checkpoints stay under ignored `data/results/benchmark/`.

> **Checkpoint:** both context variants retain inspectable outcomes with the same prepared images. This checks output behavior, not clinical accuracy.

> **Recovery:** preserve failures and unavailable predictions. Check raw output, valid image bounds and label coverage. Start a new invocation for changed configuration. Do not tune on internal-test or sealed data or expand to full sweeps merely because JSON parses.

Queue work. Keep its status.

> **Research use only.** This self-made tool may be wrong, incomplete or misleading. It is not professional medical advice or a basis for diagnosis or treatment.

Prerequisite: start one local server from the repository root with `uv run thoraxbench serve`. Analysis, comparisons and acknowledged model downloads share one worker.

1. **Submit an analysis.**

   Choose a study and model, then select “Run analysis locally”. A comparison runs its available models sequentially. Other jobs can wait.

2. **Open Jobs.**

   Inspect status, current stage, queue position, server-measured elapsed time and per-model outcomes. The bar and tab title remain visible across pages. Reloading reconnects to the same server session.

3. **Cancel waiting work if needed.**

   Select Cancel on a queued row. It will not reach a model. Select Stop on running work to close its request and stop at the next stage boundary. The provider may continue computing in the background.

4. **Open the completed result.**

   Use Open result to inspect findings and download reports. Failures, abstentions and completed comparison steps stay visible. They are not accuracy evidence.

Understand elapsed time and retention

Job elapsed uses the server monotonic clock since worker start. It freezes at completion. It differs from engine stages, queue waiting and rough estimates. The queue permits eight waiting jobs and retains 32 terminal jobs. Identical active analyses follow the existing job.

> **Checkpoint:** start the synthetic demo, open About, reload, then open Jobs. The same job remains visible. Cancel a queued job or open a completed report.

> **Recovery:** server restart loses in-memory jobs and registered download links. Saved report files remain in `data/results/ui/`. Open those files directly or submit a new job. Do not start several server workers against one GPU.